

$\text{Ctrl-}R_z(\theta)$ は $R_z(\theta/2)$ 回転ゲートと **CNOT** ゲートを組み合わせて構成します。コントロール量子ビットが $|0\rangle$ のときには、 $\theta/2$ 回転と $-\theta/2$ 回転とが相殺し、 $|1\rangle$ のときにはターゲット量子ビット $|\psi\rangle$ は

$$|\psi\rangle \xrightarrow{R_z(\frac{\theta}{2})} R_z(\frac{\theta}{2})|\psi\rangle \xrightarrow{\text{CNOT}} R_z(-\frac{\theta}{2})|\psi\rangle \xrightarrow{R_z(-\frac{\theta}{2})} R_z(-\theta)|\psi\rangle$$

と $\theta/2 - (-\theta/2) = \theta$ 回転されるカラクリです。 $\text{Ctrl-}R_z(\theta)$ ゲートを使うと、 $Z_0 Z_1$ の項の制御ゲート化 ($\text{Ctrl-}R_{zz}(\theta)$ ゲート) を構築できます。前節で紹介したように Z と X や Y の変換公式 $HZH = X$ や $SHZHS^\dagger = Y$ を使って $\text{Ctrl-}R_{xx}(\theta)$ や $\text{Ctrl-}R_{yy}(\theta)$ も作れます。

Hartree-Fock 法で求めた基底状態 $|\psi_{\text{HF}}\rangle := |01\rangle$ を初期状態として、反復的位相推定法 (3.4.2 項) で補助量子ビットに位相を 1 桁ずつ取り出すことを考えます。量子化学計算の定量性の目安として、化学的精度 (Chemical accuracy) が用いられます。これはエネルギー固有値をおよそ 2.5 kJ/mol (原子単位系で 1.0×10^{-3} Hartree) の精度で推定することに相当します³⁷⁾。

量子位相推定サブルーチンを用いた量子化学計算でこの精度を達成するには、主に 2 つのパラメータを考慮する必要があります。まず、量子位相推定サブルーチンの反復回数を考えます。(1 - ε) の成功確率で位相を L ビット精度まで求めるための反復回数は

$$L + \left\lceil \log_2 \left(2 + \frac{1}{2\varepsilon} \right) \right\rceil$$

なので^{1, 2)}、成功確率が 50% を上回る (何度か測定して多数決によって正しく 0 か 1 を決めることが可能) には

$$-\log_2(1.0 \times 10^{-3}) + \log_2 3 \approx 11.55$$

より、12 回程度の反復が必要となります (シミュレータによる試行⁵⁴⁾ では反復回数を変化させたときの精度の変化が調べられています)。

また、 Trotter 展開の次数 n も推定結果の精度を決める重要な要素です。化学的精度を達成するために必要な n は、対象とする分子や基底関数セットによって異なりますが、このケース (水素分子を STO-6G 基底で扱う) では $n = 1$ で十分であることがわかっています⁵⁵⁾。

4.3 探索と最適化

4.3.1 概要

特定の制約条件を満たす答えを見つけ出す探索は、最適化問題の解を求めるなど汎用的な問題解決手法です⁵⁶⁻⁵⁸⁾。現実の問題では、答え (ゴール) がわかっていたり、答えが正しいかどうかの判定は簡単だったりしても、問題の“解き方”がわからない場合があります。探索アルゴリズムは解法が不明の場合でも、試行錯誤などで解を求めます。

問題を解く最もシンプルで汎用な方法は、すべての可能な入力について解かどうかをしらみつぶしに確認する力まかせ探索です。しかし、可能な組合せの数は一般に膨大となり、すぐに手に負えなくなります。例えば、チェスの全局面はおよそ 10^{120} 通りの組合せとなり、すべての可能性を探索することは事実上不可能です。

そこで、多くの場合、近似アルゴリズムやヒューリスティクスにより計算コストを削減しながら近似解を探します。力まかせ探索アルゴリズムは、考えるすべての解の集合である探索空間の知識を全く用いない方法ですが、多くの近似アルゴリズムは探索空間の構造の知識を使って計算コストを削減します。

線形探索アルゴリズムは、答えが見つかるまで単純にリスト上の各要素を順に調べていきます。計算量はリスト上のアイテムの数 N に対して $O(N)$ です。より高速な二分探索アルゴリズム (計算量は $O(\log N)$) も知られています^{*12)}。root ノードからの階層数 (木の高さ) を自動的に小さく維持する平衡二分探索木という特別なデータ構造で用意できれば、 $O(\log N)$ での検索のほか、リストへの挿入や削除も $O(\log N)$ で行えます (最悪ケースでは $O(N)$)。

ハッシュテーブルはキーの要約であるハッシュ値を利用することで、リスト探索や追加をデータサイズによらない定数時間 $O(1)$ で実行します (ハッシュ値の衝突などがあると最悪の場合 $O(N)$)。線形探索や二分探索などでは若干のコスト追加で与えられたキー以下 (あるいは以上) のすべての値を探す範囲探索が可能

*12 探索すべきデータが多いほど線形探索より有利になりますが、リストを探索前にソートしておく必要があり、ランダムアクセス可能であることも条件です。